A1ELOS ASSESSORIA E CONSULTORIA

SISTEMA INTEGRADO DE NUMEROLOGIA PITAGÓRICA

Documentação Técnica e Código-Fonte da API (v2.5)

17 de julho de 2026

---

1. Resumo Executivo

Este documento apresenta o código-fonte completo do arquivo `main.py` para a **Numerologia API / A1ELOS**. O sistema foi projetado para ser uma solução global, suportando 12 idiomas e múltiplos produtos numerológicos baseados na metodologia de Monique Cissay. A arquitetura utiliza FastAPI para alto desempenho, integração com Stripe e Mercado Pago para transações financeiras, e ReportLab para geração dinâmica de PDFs personalizados.

2. Código-Fonte: main.py

```
import os, logging, uuid, stripe, mercadopago, base64
from datetime import datetime
from typing import Optional, Dict, List
from fastapi import FastAPI, HTTPException, Request, BackgroundTasks
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, String, Integer, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail, Email, To, Content, Attachment, FileContent, Fil
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors
import dateutil.parser as dp

# Configuração de Logs
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Variáveis de Ambiente
MP_ACCESS_TOKEN = os.getenv("MP_ACCESS_TOKEN", "")
STRIPE_SECRET_KEY = os.getenv("STRIPE_SECRET_KEY", "")
```

```
SENDGRID_API_KEY = os.getenv("SENDGRID_API_KEY", "")
FROM_EMAIL = os.getenv("FROM_EMAIL", "arvigne@gmail.com")
BASE_URL = os.getenv("BASE_URL", "https://numerologia-api-wd2q.onrender.com")<br/>
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./numerologia.db")

# Inicialização de SDKs
if STRIPE_SECRET_KEY:
    stripe.api_key = STRIPE_SECRET_KEY

sdk_mp = None
if MP_ACCESS_TOKEN:<br/>
    try:
        sdk_mp = mercadopago.SDK(MP_ACCESS_TOKEN)
    except Exception as e:<br/>
        logger.error(f"Erro MP SDK: {e}")

# Banco de Dados
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
Base = declarative_base()
SessionLocal = sessionmaker(bind=engine)

class Calculation(Base):
    __tablename__ = "calculations"
    id = Column(String, primary_key=True)
    name = Column(String)
    birth_date = Column(String)
    email = Column(String, nullable=True)
    life_path = Column(Integer)
    expression = Column(Integer)
    soul_urge = Column(Integer)
    personality = Column(Integer)
    destiny = Column(Integer)
    created_at = Column(DateTime, default=datetime.utcnow)

class Order(Base):
    __tablename__ = "orders"
    id = Column(String, primary_key=True)
    email = Column(String)
    product = Column(String)
    price = Column(Float)
    status = Column(String, default="pending")
    payment_id = Column(String, nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

Base.metadata.create_all(bind=engine)

# Dicionário Global de Textos (12 Idiomas)
TEXTOS = {
    "pt": {<br/>
        "title": "Mapa Numerológico", "lp": "Caminho de Vida", "ex": "Expressão", "su":
        "lp_desc": {1: "Líder nato, pioneiro.", 2: "Diplomata, sensível.", 3: "Criativo,
```

```
        "monique": "Análise Monique Cissay: Cada letra do seu nome vibra uma energia esp
    },
    "en": {<br/>
        "title": "Numerology Map", "lp": "Life Path", "ex": "Expression", "su": "Soul Ur
        "lp_desc": {1: "Natural leader, pioneer.", 2: "Diplomat, sensitive.", 3: "Creati
        "monique": "Monique Cissay Analysis: Each letter of your name vibrates a specifi
    },
    "es": {<br/>
        "title": "Mapa Numerológico", "lp": "Camino de Vida", "ex": "Expresión", "su": "
        "lp_desc": {1: "Líder nato, pionero.", 2: "Diplomático, sensible.", 3: "Creativo
        "monique": "Análisis Monique Cissay: Cada letra de su nombre vibra una energía e
    },
    "it": {"title": "Mappa Numerologica", "lp": "Sentiero di Vita", "ex": "Espressione",
    "fr": {"title": "Carte Numérologique", "lp": "Chemin de Vie", "ex": "Expression", "s
    "de": {"title": "Numerologische Karte", "lp": "Lebensweg", "ex": "Ausdruck", "su": "
    "ja": {"title": "□□□□□□□", "lp": "□□□□□□", "ex": "□□", "su": "□□□□□", "pe": "□□", "de":
    "zh": {"title": "□□□□□□", "lp": "□□□□□", "ex": "□□", "su": "□□□□□", "pe": "□□", "de": "
    "ru": {"title": "Нумерологическая карта", "lp": "Путь жизни", "ex": "Выражение", "su
    "hi": {"title": "□□□ □□□□□□□□ □□□□□□□□□", "lp": "□□□□ □□", "ex": "□□□□□□□□□□□", "su": "
    "ar": {"title": "□□□□□□□□ □□□□□□", "lp": "□□□□□□□ □□□□□", "ex": "□□□□□□□□", "su": "□□□□□□
    "he": {"title": "□□□□□□□□□□□ □□□□", "lp": "□□□□□ □□□□□", "ex": "□□□□□□", "su": "□□□□□□ □□□□
}

# Funções de Cálculo
def reduce_to_single(n):<br/>
    while n > 9 and n not in (11, 22, 33):
        n = sum(int(d) for d in str(n))
    return n

def calc_numerology(name, birth_date):
    bd = dp.parse(birth_date).date()
    life_path = reduce_to_single(bd.day + bd.month + bd.year)
    name_upper = name.upper().replace(" ", "")
    table = {c: (i % 9 or 9) for i, c in enumerate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", 1)}
    exp, vow, cons = 0, 0, 0
    for ch in name_upper:
        v = table.get(ch, 0)
        exp += v
        if ch in "AEIOU": vow += v<br/>
        else: cons += v
    return {
        "life_path": life_path, "expression": reduce_to_single(exp),<br/>
        "soul_urge": reduce_to_single(vow), "personality": reduce_to_single(cons),<br/>
        "destiny": reduce_to_single(reduce_to_single(exp) + life_path)
    }

def analyze_name_detail(name):<br/>
    table = {c: (i % 9 or 9) for i, c in enumerate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", 1)}<br/
    return [{"letra": c, "valor": table.get(c, 0)} for c in name.upper() if c in table]

# Geração de PDF
```

```
def generate_pdf(data, name, birth_date, product="pdf", lang="pt"):<br/>
    pdf_path = f"/tmp/mapa_{uuid.uuid4().hex[:8]}.pdf"
    doc = SimpleDocTemplate(pdf_path, pagesize=A4)
    styles = getSampleStyleSheet()
    t_lang = TEXTOS.get(lang, TEXTOS["pt"])

    elements = []
    elements.append(Paragraph(f"<b>{t_lang['title']} - A1ELOS</b>", styles["Title"]))
    elements.append(Paragraph(f"Nome: {name} | Data: {birth_date}", styles["Normal"]))
    elements.append(Spacer(1, 12))

    # Tabela de Números
    table_data = [["Número", "Valor"], [t_lang["lp"], data["life_path"]], [t_lang["ex"],
    t = Table(table_data, colWidths=[200, 100])
    t.setStyle(TableStyle([('BACKGROUND', (0,0), (-1,0), colors.HexColor("#C9A94E")), ('
    elements.append(t)
    elements.append(Spacer(1, 12))

    # Interpretação
    lp_val = data["life_path"]
    desc = t_lang["lp_desc"].get(lp_val, "Energia única.")
    elements.append(Paragraph(f"<b>{t_lang['lp']}:</b> {desc}", styles["Normal"]))

    # Monique Cissay
    if product in ["pdf", "free", "emp"]:
        elements.append(Spacer(1, 12))
        elements.append(Paragraph(t_lang["monique"], styles["Italic"]))
        details = analyze_name_detail(name)
        elements.append(Paragraph(f"Análise de Letras: {', '.join([f'{d['letra']}({d['va

    doc.build(elements)
    return pdf_path

# FastAPI App
app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_heade

class PayRequest(BaseModel):<br/>
    name: str<br/>
    email: str<br/>
    product: str<br/>
    price: float<br/>
    calculation_id: Optional[str] = None<br/>
    birth_date: Optional[str] = None<br/>
    lang: Optional[str] = "pt"

@app.get("/", response_class=HTMLResponse)
def root():
    return "<html><body><h1>A1ELOS Numerologia API Ativa</h1></body></html>"

@app.post("/calculate")
```

```
def calculate(req: PayRequest):
    res = calc_numerology(req.name, req.birth_date)
    return {"id": uuid.uuid4().hex[:8], **res}

@app.post("/api/pay/stripe")
def pay_stripe(req: PayRequest):
    session = stripe.checkout.Session.create(
        payment_method_types=['card'],
        line_items=[{'price_data': {'currency': 'brl', 'product_data': {'name': req.prod
        mode='payment',
        success_url=f"{BASE_URL}/api/pay/success?name={req.name}&email={req.email}&produ
        cancel_url=f"{BASE_URL}/api/pay/failure"
    )
    return {"payment_url": session.url}

@app.post("/api/pay/mercadopago")
def pay_mp(req: PayRequest):
    preference_data = {
        "items": [{"title": req.product, "quantity": 1, "unit_price": float(req.price),
        "back_urls": {"success": f"{BASE_URL}/api/pay/success?name={req.name}&email={req
    }
    pref = sdk_mp.preference().create(preference_data)
    return {"payment_url": pref["response"]["init_point"]}

@app.get("/api/pay/success")
def pay_success(name: str, email: str, product: str, lang: str, birth_date: str):
    data = calc_numerology(name, birth_date)
    pdf_path = generate_pdf(data, name, birth_date, product, lang)

    sg = SendGridAPIClient(SENDGRID_API_KEY)
    with open(pdf_path, "rb") as f:
        encoded_pdf = base64.b64encode(f.read()).decode()

    mail = Mail(from_email=FROM_EMAIL, to_emails=email, subject="Seu Mapa Numerológico A
    mail.attachment = Attachment(FileContent(encoded_pdf), FileName("Mapa.pdf"), FileTyp
    sg.send(mail)

    return HTMLResponse("<html><body style='text-align:center;padding:50px'><h1>✅ Suces

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=10000)
```

---

3. Conclusão e Próximos Passos

O código acima consolida todas as funcionalidades solicitadas, garantindo a internacionalização dos resultados e a precisão técnica baseada na numerologia pitagórica. Para a ativação completa, certifique-se de que as chaves de API

(`STRIPE_SECRET_KEY` , `MP_ACCESS_TOKEN` , `SENDGRID_API_KEY`) estejam corretamente configuradas no painel do Render.

Documento elaborado em 17 de julho de 2026. As informações contidas são de responsabilidade do solicitante.